SSO Implementation Guide

Single Sign-On (SSO) between **Tutoring App** and **Academy App** using secure cookies.

How It Works

A child logged into the Tutoring app can click a button and instantly access the Academy app — fully authenticated, no credentials required.

```
Child in Tutoring App
         ↓
   Click "Login to Academy"
         ↓
Tutoring backend sets secure cookie
         ↓
Browser redirects to Academy app
         ↓
Academy backend reads cookie & logs child in
         ↓
Child lands on Academy home page ✓
```

Why Cookies?

Secure & Invisible

- Token never appears in the URL
- HTTP-only flag prevents JavaScript access (XSS protection)
- Automatically sent by browser — no manual handling needed
- Works seamlessly across `evangadi.com` and `tutoring.evangadi.com`

Short-Lived

- Bridge cookie expires in 30 seconds
- Deleted immediately after use
- Minimizes attack window

The Flow — Step by Step

1. Child Clicks "Login to Academy"

File: `tutoring-app/frontend/src/pages/ChildHome.jsx`

When the child clicks the button, the frontend sends their existing session token to the Tutoring backend:

```
const token = localStorage.getItem("childToken");

fetch("http://localhost:5001/api/child/sso-token", {
  method: "POST",
  headers: { Authorization: `Bearer ${token}` },
  credentials: "include", // Important: enables cookie handling
});
```

---

2. Tutoring Backend Creates Bridge Token

File: `tutoring-app/backend/server.js` → `POST /api/child/sso-token`

The backend:

1. Verifies the child's session token
2. Fetches child + parent data from database
3. Creates a **tagged email** for the child:

```
Parent email:    john@example.com
Child username:  james.tyler210
Tagged email:    john+james.tyler210@example.com
```

4. Signs a **bridge token** with `SSO_BRIDGE_SECRET` (expires in 30 seconds):

```
{
  "id": 3,
  "username": "james.tyler210",
  "name": "James",
  "email": "john+james.tyler210@example.com"
}
```

5. Sets the token as a **secure cookie**:

```
res.cookie("sso_bridge_token", ssoToken, {
  httpOnly: true, // JavaScript can't access it
  secure: true, // HTTPS only (in production)
  sameSite: "lax", // Allows cross-subdomain
  domain: ".evangadi.com", // Shared across subdomains
  maxAge: 30000, // 30 seconds
  path: "/",
});
```

Why a tagged email?

Each child gets a unique identity in the Academy app without needing their own email address. The format `parent+childusername@domain.com` is standard email syntax — most providers support it.

---

3. Browser Redirects to Academy App

File: `tutoring-app/frontend/src/pages/ChildHome.jsx`

After the cookie is set, the frontend redirects:

```
window.location.href = "http://localhost:5173/sso";
```

Notice: No token in the URL! The cookie travels automatically with the request.

---

4. Academy App Reads the Cookie

File: `academy-app/frontend/src/pages/SSOLogin.jsx`

The SSO landing page immediately calls the Academy backend:

```
fetch("http://localhost:5000/api/sso/login", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  credentials: "include", // Browser sends the cookie automatically
});
```

---

5. Academy Backend Verifies & Logs Child In

File: `academy-app/backend/server.js` → `POST /api/sso/login`

The backend:

1. Reads the cookie: `req.cookies.sso_bridge_token`
2. Verifies it using `SSO_BRIDGE_SECRET`
3. Extracts `name` and `email` from the token
4. Checks if the child already exists in the `users` table:
    - **Exists** → use that record
    - **Doesn't exist** → auto-create a new user with a random password (child can only log in via SSO)
5. Issues a normal Academy session token (signed with Academy's own `JWT_SECRET`)
6. **Clears the bridge cookie** (one-time use)
7. Returns the session token in the response body (same as normal login)

```
// Clear bridge cookie after use
res.clearCookie("sso_bridge_token", {
  httpOnly: true,
```

```
    secure: process.env.NODE_ENV === "production",
    sameSite: "lax",
    domain: process.env.NODE_ENV === "production" ? ".evangadi.com" : "localhost",
    path: "/",
  });

  // Return token in response body (consistent with normal login)
  res.json({
    token: sessionToken,
    user: { id: user.id, name: user.name, email: user.email },
  });
```

---

6. Child Lands on Academy Home Page

File: `academy-app/frontend/src/pages/SSOLogin.jsx`

The frontend:

1. Receives the session token
2. Stores it in `localStorage` (same as a normal login)
3. Redirects to `/` — the Academy home page

```
localStorage.setItem("token", data.token);
localStorage.setItem("user", JSON.stringify(data.user));
navigate("/");
```

The child sees: **"Welcome, James 👋"**

Security Features

Feature	Implementation
No URL exposure	Bridge token in cookie, not query parameter
XSS protection	`httpOnly: true` — JavaScript cannot access the cookie
CSRF protection	`sameSite: 'lax'` — prevents cross-site attacks
HTTPS enforcement	`secure: true` in production
Short lifespan	30-second expiry + immediate deletion after use
Separate secrets	Bridge uses `SSO_BRIDGE_SECRET`, apps use their own secrets
One-time use	Cookie cleared immediately after verification
Consistent auth flow	Academy session handled same as normal login (localStorage)
Auto-provisioning	Child accounts created automatically with secure placeholders

Feature	Implementation
Cross-subdomain support	Cookie domain `.evangadi.com` works for all subdomains
CSRF protection	`sameSite: 'lax'` — prevents cross-site attacks
HTTPS enforcement	`secure: true` in production
Short lifespan	30-second expiry + immediate deletion after use
Separate secrets	Bridge uses `SSO_BRIDGE_SECRET`, apps use their own secrets
One-time use	Cookie cleared immediately after verification
Auto-provisioning	Child accounts created automatically with secure placeholders
Cross-subdomain support	Cookie domain `.evangadi.com` works for all subdomains

Configuration

Environment Variables

Both apps need `SSO_BRIDGE_SECRET` in their `.env` files with the **same value**:

tutoring-app/backend/.env

```
JWT_SECRET=tutoring_secret_key
SSO_BRIDGE_SECRET=sso_shared_bridge_secret
PORT=5001
```

academy-app/backend/.env

```
JWT_SECRET=academy_secret_key
SSO_BRIDGE_SECRET=sso_shared_bridge_secret
PORT=5000
```

Dependencies

Both backends need `cookie-parser`:

```
npm install cookie-parser
```

CORS Configuration

Both backends must enable credentials:

```
app.use(
  cors({
    origin: ["http://localhost:5173", "http://localhost:5174"],
    credentials: true, // Required for cookies
  }),
);
```

Local Development vs Production

Local (localhost)

```
domain: "localhost"; // Works across different ports
secure: false; // HTTP allowed
```

URLs:

- Tutoring: `http://localhost:5174`
- Academy: `http://localhost:5173`

Production

```
domain: ".evangadi.com"; // Note the leading dot
secure: true; // HTTPS required
```

URLs:

- Tutoring: `https://tutoring.evangadi.com`
- Academy: `https://evangadi.com`

Important: Set `NODE_ENV=production` in both `.env` files for production deployment.

Testing the Implementation

1. **Start both backends:**

```
cd tutoring-app/backend && npm run dev
cd academy-app/backend && npm run dev
```

2. **Start both frontends:**

```
cd tutoring-app/frontend && npm run dev
cd academy-app/frontend && npm run dev
```

3. **Test the flow:**
    - Register a parent in Tutoring app
    - Add a child profile
    - Log in as the child
    - Click "Login to Academy"
    - Verify you land on Academy home page
4. **Verify security:**
    - Check the URL — no token visible
    - Open DevTools → Application → Cookies
    - See `sso_bridge_token` briefly appear and disappear
    - Confirm `HttpOnly` and `Secure` flags are set

Files Modified

Backend

File	Changes
`tutoring-app/backend/package.json`	Added `cookie-parser` dependency
`tutoring-app/backend/server.js`	Added cookie middleware, SSO endpoint sets cookie
`academy-app/backend/package.json`	Added `cookie-parser` dependency
`academy-app/backend/server.js`	Added cookie middleware, SSO endpoint reads/clears cookie
`tutoring-app/backend/.env`	Added `SSO_BRIDGE_SECRET`
`academy-app/backend/.env`	Added `SSO_BRIDGE_SECRET`

Frontend

File	Changes
`tutoring-app/frontend/src/pages/ChildHome.jsx`	Added `credentials: 'include'`, removed URL token
`academy-app/frontend/src/pages/SSOLogin.jsx`	Removed URL parsing, added `credentials: 'include'`

Troubleshooting

Cookie not being sent?

- Verify `credentials: 'include'` in all fetch calls
- Check CORS has `credentials: true`

- Ensure cookie domain matches (use `localhost` in dev)

Token verification fails?

- Confirm `SSO_BRIDGE_SECRET` is identical in both `.env` files
- Check token hasn't expired (30 seconds)
- Verify `cookie-parser` middleware is loaded

Cookie not visible in DevTools?

- HTTP-only cookies don't appear in JavaScript
- Check Application → Cookies in DevTools
- Cookie is deleted after use — check immediately after redirect

Production issues?

- Set `NODE_ENV=production` in both apps
- Use HTTPS (required for `secure: true`)
- Cookie domain must be `.evangadi.com` (with leading dot)